

W06 Team05 설명문서

NipsEngine - 6주차 구현 정리: 포그와 데칼

KRAFTON Jungle GameTechLab 과정에서 제작한 C++ / DirectX 11 기반 커스텀 엔진 프로젝트입니다.

이 저장소는 기존 포워드 렌더러 위에 깊이 버퍼 기반 효과를 확장하고, 프레임 흐름을 멀티 패스 구조로 정리한 6주차 작업 결과를 담고 있습니다. 핵심 목표는 기본 장면 렌더링 위에 포그, 데칼, FireBall, 아웃라인, FXAA를 자연스럽게 연결하면서도, 전체 엔진 구조는 포워드 렌더링 기반으로 유지하는 것이었습니다.

이 프로젝트에서 다루는 내용

이번 주차 구현은 다음 네 가지 주제를 중심으로 구성되어 있습니다.

- 깊이 버퍼를 이용한 후처리 포그
- 장면 깊이를 이용하는 화면 공간 볼륨 데칼
- DirectX 11 기반 멀티패스 프레임 구성
- BVH 기반 컬링과 피킹 보조 구조

즉, 먼저 장면을 일반적으로 렌더링한 뒤 그 과정에서 생성된 `SceneColor` 와 `Depth` 를 다시 활용해 후속 패스에서 추가 효과를 누적하는 구조입니다.

주요 특징

렌더링

- 정적 메시 기반 포워드 렌더링
- 지연 렌더링이 아닌 멀티패스 프레임 구성
- 전체 화면 삼각형 기반 후처리 패스
- 깊이 시각화, 아웃라인, FXAA 지원

그래픽 효과

- 지수 감쇠 기반 높이 포그
- 거리 기반 포그 감쇠
- 박스 볼륨 기반 화면 공간 데칼
- 가산 합성 기반 `FireBall` / 광원 볼륨 스타일 패스

엔진 / 에디터

- 에디터 중심 실행 경로
- 다중 뷰포트 렌더링
- 선택 마스크와 아웃라인 연동
- BVH 기반 프러스텀 컬링 및 레이 피킹 1차 후보 추리기

렌더링 구조

현재 저장소에서 실제로 활성화되는 기본 실행 경로는 에디터 렌더러입니다.

- 엔진 진입점: `NipsEngine/main.cpp`
- 런치 경로: `NipsEngine/Source/Engine/Runtime/Launch.cpp`
- 메인 루프: `NipsEngine/Source/Engine/Runtime/EngineLoop.cpp`
- 에디터 렌더 파이프라인: `NipsEngine/Source/Editor/EditorRenderPipeline.cpp`
- 핵심 렌더러: `NipsEngine/Source/Engine/Render/Renderer/Renderer.cpp`

에디터 한 프레임의 큰 흐름은 다음과 같습니다.

```
```text
```

```
UEditorEngine::Tick
```

- > FEditorRenderPipeline::Execute
- > BeginFrame
- > UseViewportRenderTargets
- > 각 뷰포트에 대해 반복:

```
→ SetSubViewport
```

```
→ CollectWorld
```

```
→ CollectGrid
```

```
→ CollectGizmo
```

```
→ CollectSelection
```

```
→ PrepareBatchers
```

```
→ Render
```

- > ImGui가 최종 뷰포트 텍스처를 화면에 표시
- > EndFrame

```
```
```

`FRenderer::Render` 내부의 패스 순서는 다음과 같습니다.

```text

Opaque

- > Decal
- > Translucent
- > Fog
- > FireBall
- > Grid
- > SelectionMask
- > PostProcessOutline
- > FXAA
- > Editor / Font / SubUV / DepthLess(Gizmo)

```

이 순서가 중요한 이유는, 현재 프로젝트가 지연 렌더러가 아니라 포워드 렌더링 기반 멀티 패스 구조이기 때문입니다. 먼저 기본 장면이 `SceneColor` 와 `Depth` 를 만든 뒤, 이후 패스들이 그 결과를 읽어 추가 효과를 엮습니다.

주요 기능 동작 방식

1. 기본 장면 렌더링

정적 메시는 `NipsEngine/Shaders/ShaderStaticMesh.hlsl` 기반 포워드 패스로 먼저 렌더링됩니다. 이 단계에서 최종 장면 색과 깊이 버퍼가 생성되며, 이후 데칼과 포그 패스는 이 깊이 정보를 재사용합니다.

관련 파일:

- `NipsEngine/Shaders/ShaderStaticMesh.hlsl`
- `NipsEngine/Source/Engine/Render/Scene/RenderCollector.cpp`
- `NipsEngine/Source/Engine/Render/Renderer/Renderer.cpp`

2. 포그 패스

포그는 메시별 셰이딩이 아니라 후처리 패스로 구현되어 있습니다.

이 패스는 다음 순서로 동작합니다.

- 깊이 버퍼 SRV에서 현재 픽셀의 깊이 값을 읽음

- 깊이로부터 현재 픽셀의 월드 위치를 복원
- 픽셀 셰이더에서 높이 기반 지수 포그를 계산
- 계산된 포그 색과 알파를 장면 색 위에 블렌딩

관련 파일:

- `NipsEngine/Shaders/Fog.hlsl`
- `NipsEngine/Source/Engine/Component/HeightFogComponent.h`
- `NipsEngine/Source/Engine/Component/HeightFogComponent.cpp`
- `NipsEngine/Source/Engine/Render/Scene/RenderCollector.cpp`

3. 데칼 패스

데칼은 박스 볼륨을 그린 뒤, 각 픽셀에서 장면 깊이를 읽어 실제 표면 위치를 복원하고, 그 위치를 데칼 로컬 공간으로 변환해 볼륨 내부 픽셀만 남기는 방식으로 구현되어 있습니다.

즉, CPU가 데칼을 받을 메시지를 직접 찾는 구조가 아니라 깊이 버퍼를 통해 현재 화면에 보이는 표면 정보를 가져와 투영하는 구조입니다.

관련 파일:

- `NipsEngine/Shaders/Decal.hlsl`
- `NipsEngine/Source/Engine/Component/DecalComponent.h`
- `NipsEngine/Source/Engine/Component/DecalComponent.cpp`
- `NipsEngine/Source/Engine/Render/Resource/MeshBufferManager.cpp`

4. FireBall 패스

`FireBall` 은 일반 메시 렌더링이 아니라 볼륨 기반 특수 효과 패스로 동작합니다.

`UFireBallComponent` 가 반지름, 강도, 감쇠 값을 가지고 있고, 렌더 수집 단계에서 `ERenderPass::FireBall` 커맨드로 변환됩니다.

이 패스는 다음 특징을 가집니다.

- 데칼과 마찬가지로 unit cube 메시지를 재사용해 볼륨을 그림
- 깊이 버퍼를 읽어 현재 화면에 실제로 보이는 표면 위치를 기준으로 효과를 계산
- `InverseClipToLocal` 로 픽셀을 FireBall 로컬 공간으로 되돌린 뒤 반지름 내부만 남김
- `Intensity`, `Radius`, `RadiusFallOff` 값을 이용해 중심부는 강하고 가장자리는 약한 발광 효과를 계산

- 최종 출력은 'Additive' 블렌드로 장면 위에 누적

즉 현재 구현의 'FireBall' 은 물리 기반 광원 시스템이라기보다, 깊이 버퍼를 활용해 장면에 더해지는 볼륨형 발광 / 라이트 볼륨 스타일 패스에 가깝습니다.

관련 파일:

- 'NipsEngine/Shaders/FireBall.hlsl'
- 'NipsEngine/Source/Engine/Component/FireBallComponent.h'
- 'NipsEngine/Source/Engine/Component/FireBallComponent.cpp'
- 'NipsEngine/Source/Engine/Render/Scene/RenderCollector.cpp'
- 'NipsEngine/Source/Engine/Render/Renderer/Renderer.cpp'

5. 후처리와 에디터 오버레이

장면과 효과 패스 이후에는 다음 단계가 이어집니다.

- 선택 마스크 생성
- 아웃라인 확장
- FXAA 적용
- 그리드, 기즈모, 텍스트, 'SubUV' 같은 에디터 오버레이 렌더링

관련 파일:

- 'NipsEngine/Shaders/SelectionMask.hlsl'
- 'NipsEngine/Shaders/OutlinePostProcess.hlsl'
- 'NipsEngine/Shaders/ShaderFXAA.hlsl'

공간 구조와 피킹

이 엔진은 월드 단위 공간 인덱스로 BVH를 사용합니다.

현재 BVH가 담당하는 역할:

- 렌더 커맨드 수집 시 프러스텀 컬링
- 에디터 피킹을 위한 광선 질의 1차 후보 추리기
- 정밀 판정 이전 후보 수 축소

현재 BVH가 담당하지 않는 역할:

- 데칼 투영 대상을 직접 찾는 작업
- OBBTree 기반 계층 탐색

- 활성 기본 렌더 경로에서의 SAT 기반 OBB 교차 테스트

관련 파일:

- `NipsEngine/Source/Engine/Spatial/WorldSpatialIndex.h`
- `NipsEngine/Source/Engine/Spatial/WorldSpatialIndex.cpp`
- `NipsEngine/Source/Engine/Spatial/BVH.h`
- `NipsEngine/Source/Engine/Spatial/BVH.cpp`
- `NipsEngine/Source/Editor/Viewport/EditorViewportClient.cpp`

프로젝트 구조

```text

NipsEngine/

+++ Shaders/                    장면, 포그, 데칼, FXAA, 아웃라인 등에 사용하는 HLSL 셰이더

+++ Source/

|    +++ Editor/                에디터 런타임, 뷰포트 로직, UI, 렌더 파이프라인

|    +++ Engine/

|        +++ Component/        씬, 메시, 데칼, 포그, Billboard, FireBall 컴포넌트

|        +++ Render/            Renderer, Render Command, Device/State 설정, 리소스 관리

|        +++ Runtime/          엔진 실행 루프와 애플리케이션 수명 주기

|        +++ Spatial/          BVH와 월드 공간 인덱스

|        +++ GameFramework/    World, Actor, 씬 구성

+++ Asset/                    메시, 텍스처, 씬, 폰트, 파티클 리소스

+++ NipsEngine.vcxproj        Visual Studio 프로젝트 파일

...

## ## 빌드 및 실행

### ### 요구 사항

- Windows 환경
- Visual Studio 2022
- `v143` MSVC 도구 집합

- Windows 10 SDK

### ### 프로젝트 열기

1. `NipsEngine.sln` 을 Visual Studio에서 엽니다.
2. `NipsEngine` 타겟을 빌드합니다.
3. Visual Studio에서 실행합니다.

프로젝트 파일이 필요하면 다음 배치 파일로 다시 생성할 수 있습니다.

```
```bat
```

```
GenerateProjectFiles.bat
```

```
```
```

### ### 기본 실행 모드

기본 Visual Studio 빌드 설정에는 `WITH\_EDITOR=1` 이 정의되어 있으므로, 일반 실행 시 에디터 경로로 시작합니다.

### ## 처음 읽을 때 추천하는 순서

처음 저장소를 읽을 때는 아래 순서가 가장 빠릅니다.

1. `NipsEngine/Source/Editor/EditorRenderPipeline.cpp`
2. `NipsEngine/Source/Engine/Render/Renderer/Renderer.cpp`
3. `NipsEngine/Source/Engine/Render/Scene/RenderCollector.cpp`
4. `NipsEngine/Shaders/Fog.hlsl`
5. `NipsEngine/Shaders/Decal.hlsl`
6. `NipsEngine/Source/Engine/Spatial/WorldSpatialIndex.cpp`
7. `NipsEngine/Source/Engine/Spatial/BVH.cpp`

### ## 현재 범위와 참고 사항

- 포그는 전용 수집 경로를 통해 렌더러에 전달되며, 포그 관련 에디터 모드와 포그 액터 설정을 함께 보는 것이 가장 이해하기 쉽습니다.
- 데칼은 현재 화면에 기록된 깊이 값을 기준으로 작동하므로, 이미 깊이 버퍼에 존재하는 표면에만 투영됩니다.
- FireBall 은 `ERenderPass::FireBall` 로 별도 실행되며, depth 를 읽어 로컬 볼륨 내부 픽셀만 남긴 뒤 가산 합성으로 누적됩니다.
- `Source/Engine/Render/Renderer/OcclusionCulling/` 아래에 실험적인 오클루전 컬링 코드가 존재하지만, 이 README는 현재 기본 활성 렌더 경로만 기준으로 설명합니다.

니다.

## ## 요약

이 저장소는 작은 DirectX 11 포워드 렌더러를 기반으로, 깊이 버퍼를 재활용하는 그래픽 효과를 어떻게 확장할 수 있는지 보여 주는 실전 예제입니다.

이번 주치의 핵심 성과는 단순히 포그나 데칼 기능 하나를 추가한 것이 아닙니다. 더 중요한 변화는 다음과 같습니다.

- 기본 장면을 먼저 렌더링하고
- 그 과정에서 생성된 깊이 버퍼를 공용 데이터처럼 재사용하며
- 데칼과 FireBall 같은 볼륨 패스, 그리고 후처리 패스를 연결하고
- 에디터 도구와 공간 인덱스 구조를 같은 프레임 구조 안에 통합했다는 점입니다

즉 이 프로젝트는 다음 주제를 공부하는 사람에게 좋은 참고 자료가 됩니다.

- DirectX 11 렌더 패스 구성
- 전체 화면 후처리 구현
- 깊이 기반 월드 위치 복원
- 화면 공간 데칼 투영
- BVH 기반 엔진 워크플로우